

Backend & Database Integration

Cheatsheet — Milestone 9

1. Node.js & Express Basics

Project Setup

npm init -y দিয়ে package.json তৈরি হয়। Express হলো server framework, Nodemon code change হলে automatically server restart করে।

```
npm init -y
npm install express
npm install --save-dev nodemon
```

Basic Express Server

express.json() middleware ছাড়া POST request-এর body পড়া যাবে না। listen() দিয়ে server নির্দিষ্ট port-এ চালু হয়।

```
const express = require('express');
const app = express();
app.use(express.json());

app.listen(3000, () => console.log('Server running on port 3000'));
```

GET Route

req মানে request (client থেকে আসা), res মানে response (server থেকে পাঠানো)।

```
app.get('/users', (req, res) => {
  res.json({ message: 'Hello from server' });
});
```

POST Route

Client form বা fetch দিয়ে data পাঠালে সেটা req.body তে পাওয়া যায়। এখান থেকে database-এ save করা হয়।

```
app.post('/users', (req, res) => {
  const data = req.body;
  res.json({ received: data });
});
```

2. MongoDB Atlas Setup

Atlas হলো MongoDB-এর cloud version। Local-এ কিছু install না করেই internet-এ database রাখা যায়।

- mongodb.com এ গিয়ে Free cluster তৈরি করো
- Database user যোগ করো (username + password)
- IP whitelist করো: 0.0.0.0/0 (সব IP allow)
- Connection string copy করো

```
mongodb+srv://<user>:<password>@cluster.mongodb.net/myDB
```

⚠️ Production-এ নির্দিষ্ট IP দেওয়া উচিত, 0.0.0.0/0 শুধু development-এ ব্যবহার করো।

3. MongoDB Connection

async/await দিয়ে MongoDB-এর সাথে connection করা হয় কারণ এটি একটি asynchronous operation। try-catch দিয়ে connection fail হলে error handle করা হয়। URI সবসময় .env file-এ রাখতে হবে।

```
const { MongoClient } = require('mongodb');

const uri = process.env.MONGODB_URI;
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    console.log('Connected to MongoDB');
  } catch (err) {
    console.error(err);
  }
}

run();
```

4. CRUD Operations

Create — insertOne

Database-এ নতুন একটি document তৈরি করে। MongoDB automatically একটি unique `_id` যোগ করে দেয়।

```
const result = await collection.insertOne({ name: 'Alice', age: 25 });
```

Read — find all

`.toArray()` না দিলে cursor আসে, actual data আসে না।

```
const users = await collection.find().toArray();
```

Read by ID — findOne

MongoDB-এর ID সাধারণ string না, এটি ObjectId type। তাই `new ObjectId(id)` দিয়ে convert করতে হয়, নইলে কিছুই খুঁজে পাবে না।

```
const { ObjectId } = require('mongodb');
const user = await collection.findOne({ _id: new ObjectId(id) });
```

Update — updateOne

`$set` ব্যবহার না করলে পুরো document replace হয়ে যাবে।

```
await collection.updateOne(
  { _id: new ObjectId(id) },
  { $set: { name: 'Bob' } }
);
```

Delete — deleteOne

নির্দিষ্ট ID-র document permanently delete করে। Undo করা যায় না, তাই client side-এ confirm নেওয়া ভালো।

```
await collection.deleteOne({ _id: new ObjectId(id) });
```

5. MongoDB Query Operators

Comparison Operators

SQL-এর WHERE clause-এর মতোই কাজ করে। Number, date, string সব ধরনের comparison করা যায়।

\$eq	→ equal	{ age: { \$eq: 25 } }
\$ne	→ not equal	{ age: { \$ne: 25 } }
\$gt	→ greater than	{ age: { \$gt: 18 } }
\$gte	→ greater or equal	{ age: { \$gte: 18 } }
\$lt	→ less than	{ age: { \$lt: 60 } }
\$lte	→ less or equal	{ age: { \$lte: 60 } }

Logical Operators

\$and মানে দুটো condition-ই সত্য হতে হবে। \$or মানে যেকোনো একটি সত্য হলেই চলবে।

```
// AND
collection.find({ $and: [{ age: { $gte: 18 } }, { city: 'Dhaka'
}] });

// OR
collection.find({ $or: [{ age: 18 }, { city: 'Dhaka' }] });
```

Array Operators

\$in মানে যেকোনো একটি value match করলেই হবে। \$all মানে সবগুলো থাকতে হবে। \$size দিয়ে array-র exact length check করা হয়।

```
collection.find({ tags: { $in: ['travel', 'food'] } });
collection.find({ tags: { $all: ['travel', 'food'] } });
collection.find({ tags: { $size: 3 } });
```

Element Operators

\$exists দিয়ে check করা হয় field টি আদৌ আছে কিনা। \$type দিয়ে data type check করা যায়।

```
collection.find({ phone: { $exists: true } });
collection.find({ age: { $type: 'number' } });
```

6. MongoDB Update Operators

এই operators গুলো পুরো document replace না করে শুধু দরকারি অংশটুকু পরিবর্তন করে।

```
{ $set: { name: 'Alice' } } // নির্দিষ্ট field update করে
{ $inc: { age: 1 } } // numeric value বাড়ায় বা কমায়
{ $push: { tags: 'new' } } // array-তে নতুন item যোগ করে
{ $pull: { tags: 'old' } } // array থেকে নির্দিষ্ট item সরায়
{ $addToSet: { tags: 'unique' } } // duplicate ছাড়া array-তে যোগ করে
```

7. Projection, Sort, Limit & Skip

Projection দিয়ে শুধু দরকারি field গুলো fetch করা হয় – unnecessary data transfer কমে, performance ভালো হয়। sort(-1) মানে descending। skip().limit() দিয়ে pagination তৈরি করা হয়।

```
// Projection – শুধু name ও age নিবে, _id বাদ
collection.find({}, { projection: { name: 1, age: 1, _id: 0 }
});

// Sort – বড় থেকে ছোট (descending)
collection.find().sort({ age: -1 });

// Limit – সর্বোচ্চ 10টি document
collection.find().limit(10);

// Pagination – page 2 দেখাতে
collection.find().skip(10).limit(10);
```

8. Aggregation Pipeline

Aggregation হলো data processing-এর powerful tool। Pipeline মানে data একটার পর একটা stage-এর মধ্য দিয়ে যায়।

```
collection.aggregate([
  { $match: { city: 'Dhaka' } },           // filter
  { $group: { _id: '$city',                 // group করে count
    total: { $sum: 1 } } },
  { $project: { city: '$_id',               // output shape
    total: 1, _id: 0 } }
]);
```

Advanced — \$lookup & \$unwind

\$lookup হলো SQL-এর JOIN-এর মতো — দুটো collection-এর data একসাথে আনে। \$unwind array-কে আলাদা document-এ ভেঙে দেয়।

```
{ $lookup: {
  from: 'orders',
  localField: '_id',
  foreignField: 'userId',
  as: 'orders'
}}
{ $unwind: '$orders' }
```

9. Authentication with BetterAuth

BetterAuth একটি modern authentication library যা sign up, sign in, Google login সব একসাথে handle করে। Password hashing, session management নিজে থেকেই করে।

```
npm install better-auth

import { betterAuth } from 'better-auth';
export const auth = betterAuth({
  database: mongodbAdapter(db),
  socialProviders: {
    google: {
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    }
  }
});
```

10. JWT Authentication

HTTPOnly Cookie-তে Store করা

httpOnly: true মানে JavaScript দিয়ে এই cookie access করা যাবে না – XSS attack থেকে সুরক্ষা দেয়।

```
res.cookie('token', token, {  
  httpOnly: true,  
  secure: true,  
  sameSite: 'none'  
});
```

Token Verify করা (Middleware)

Middleware হলো route-এর আগে চলা function। Token না থাকলে বা ভুল হলে request আটকে দেয়। Valid হলে req.user-এ decoded data রেখে next() দিয়ে পরের step-এ যায়।

```
const verifyToken = (req, res, next) => {  
  const token = req.cookies.token;  
  if (!token) return res.status(401).json({ message:  
'Unauthorized' });  
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {  
    if (err) return res.status(403).json({ message: 'Forbidden'  
  });  
  req.user = decoded;  
  next();  
});
```

Route Protect করা

verifyToken middleware দিয়ে route protect করা হয়। এখন এই route-এ valid token ছাড়া কেউ access করতে পারবে না।

```
app.get('/profile', verifyToken, (req, res) => {  
  res.json({ user: req.user });  
});
```

Logout

Cookie clear করলেই logout হয়ে যায়। Server-side-এ cookie delete করা client-side clear করার চেয়ে বেশি secure।

```
res.clearCookie('token');  
res.json({ message: 'Logged out' });
```

11. Environment Variables

Sensitive information যেমন password, secret key, database URL কখনো সরাসরি code-এ লেখা যাবে না। .env file-এ রাখতে হবে এবং .gitignore-এ add করতে হবে যাতে GitHub-এ upload না হয়।

```
# .env file  
PORT=3000  
MONGODB_URI=mongodb+srv://...  
JWT_SECRET=your_secret_key  
GOOGLE_CLIENT_ID=...  
GOOGLE_CLIENT_SECRET=...
```

```
require('dotenv').config();  
const port = process.env.PORT;
```

12. HTTP Methods — Quick Reference

Method	উদ্দেশ্য	Example Route
GET	Data fetch করতে	/users
POST	নতুন data তৈরি করতে	/users
PUT	পুরো document replace করতে	/users/:id
PATCH	আংশিক update করতে	/users/:id
DELETE	Data মুছে ফেলতে	/users/:id